Problem Solving, Problem Space & Search

AUTHOR : PROF. SANJAY KALAMDHAD

DEPARTMENT : CSBS

Introduction to Problem Solving, General problem solving

- Problem solving is a process of generating solutions from observed data.
  - a problem is characterized by a set of goals,
  - a set of objects, and
  - a set of operations.

  These could be ill-defined and may evolve during problem solving.
- **Problem solving**: The term, Problem Solving relates to analysis in AI. Problem solving may be characterized as a systematic search through a range of possible actions to reach some predefined goal or solution. Problem-solving methods are categorized ---
  - A special-purpose method is tailor-made for a particular problem, often exploits very specific features of the situation in which the problem is embedded.
  - A general-purpose method is applicable to a wide variety of problems. One General-purpose technique used in AI is 'means-end analysis': a step-by step, or incremental, reduction of the difference between current state and final goal.

Problem solving

- **Problem solving** is an important aspect of Artificial Intelligence. A problem can be considered to consist of a goal and a set of actions that can be taken to lead to the goal. At any given time, we consider the state of the search space to represent where we have reached as a result of the actions we have applied so far. For example, consider the problem of looking for a contact lens on a football field. The initial state is how we start out, which is to say we know that the lens is somewhere on the field, but we don't know where. If we use the representation where we examine the field in units of one square foot, then our first action might be to examine the square in the top-left corner of the field. If we do not find the lens there, we could consider the state now to be that we have examined the top-left square and have not found the lens. After a number of actions, the state might be that we have examined 500 squares, and we have now just found the lens in the last square we examined. This is a goal state because it satisfies the goal that we had of finding a contact lens.

Searching Solutions:

- To build a system to solve a problem:

1. Define the problem precisely
2. Analyze the problem
3. Isolate and represent the task knowledge that is necessary to solve the problem
4. Choose the best problem-solving techniques and apply it to the particular problem.

To solve the problem of playing a game, we require the rules of the game and targets for winning as well as representing positions in the game. The opening position can be defined as the **initial state** and a winning position as a **goal state**. Moves from initial state to other states leading to the goal state follow legally.

- The number of rules that are used must be minimized and the set can be created by expressing each rule in a form as possible.
- The representation of games leads to a state space representation and it is common for well-organized games with some structure.
- This representation allows for the formal definition of a problem that needs the movement from a set of initial positions to one of a set of target positions.
- It means that the solution involves using **known techniques** and a **systematic search**. This is quite a common method in Artificial Intelligence.

State Space Search

Defining the problem as State Space Search:

The state space representation forms the basis of most of the AI methods.

- Formulate a problem as a state space search by showing the legal problem states, the legal operators, and the initial and goal states.
- A state is defined by the specification of the values of all attributes of interest in the world
- An operator changes one state into the other; it has a precondition which is the value of certain attributes prior to the application of the operator, and a set of effects, which are the attributes altered by the operator The initial state is where you start
- The goal state is the partial description of the solution

Formal Description of the problem

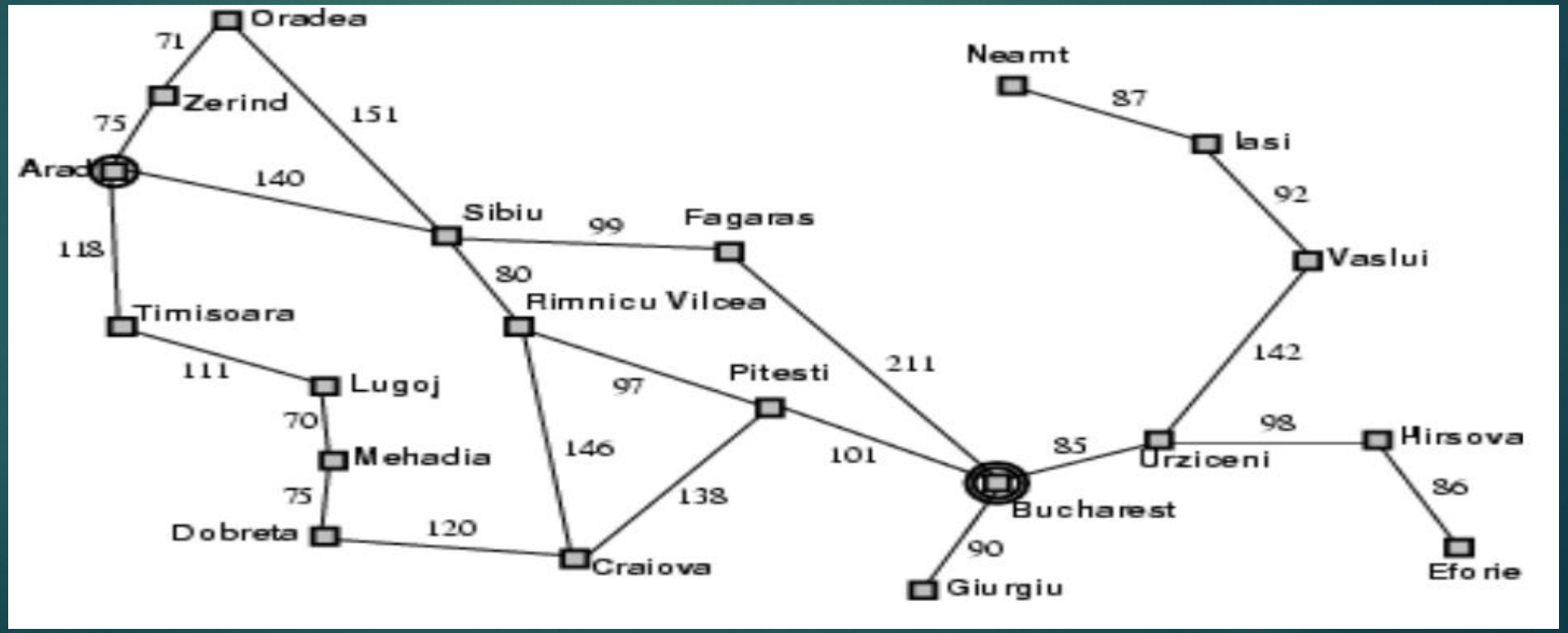
1. Define a state space that contains all the possible configurations of the relevant objects.
2. 2. Specify one or more states within that space that describe possible situations from which the problem solving process may start(initial state)
3. 3. Specify one or more states that would be acceptable as solutions to the problem.(goal states) Specify a set of rules that describe the actions(operations) available

State-Space Problem Formulation

Example: A problem is defined by four items:

1. Initial state e.g., "**Arad**—
2. Actions or successor function : S(x) = set of action–state pairs e.g., S(Arad) = {, … }
3. Goal test (or set of goal states) e.g., x = "**Bucharest**.
4. Path cost (additive) e.g., sum of distances, number of actions executed, etc. c(x,a,y) is the step cost, assumed to be ≥ 0

A solution is a sequence of actions leading from the initial state to a goal state



Water Jug Problem

- In this problem, we use two jugs called four and three; four holds a maximum of four gallons of water and three a maximum of three gallons of water. How can we get two gallons of water in the four jug?
- The state space is a set of prearranged pairs giving the number of gallons of water in the pair of jugs at any time, i.e., (four, three) where four = 0, 1, 2, 3 or 4 and three = 0, 1, 2 or 3.
- The start state is (0, 0) and the goal state is (2, n) where n may be any but it is limited to three holding from 0 to 3 gallons of water or empty. Three and four shows the name and numerical number shows the amount of water in jugs for solving the water jug problem. The major

production rules for solving this problem are shown below:

Initial condition	Goal comment
1. (four, three) if four < 4	(4, three) fill four from tap
2. (four, three) if three< 3	(four, 3) fill three from tap
3. (four, three) If four > 0	(0, three) empty four into drain
4. (four, three) if three > 0	(four, 0) empty three into drain
5. (four, three) if four + three<4	(four + three, 0) empty three into four
6. (four, three) if four + three<3	(0, four + three) empty four into three
7. (0, three) If three > 0	(three, 0) empty three into four
8. (four, 0) if four > 0	(0, four) empty four into three
9. (0, 2)	(2, 0) empty three into four
10. (2, 0)	(0, 2) empty four into three
11. (four, three) if four < 4	(4, three-diff) pour diff, 4-four, into four from three
12. (three, four) if three < 3	(four-diff, 3) pour diff, 3-three, into three from four and a solution is given below four three rule

***(Fig. 2.2** Production Rules for the Water Jug Problem)*

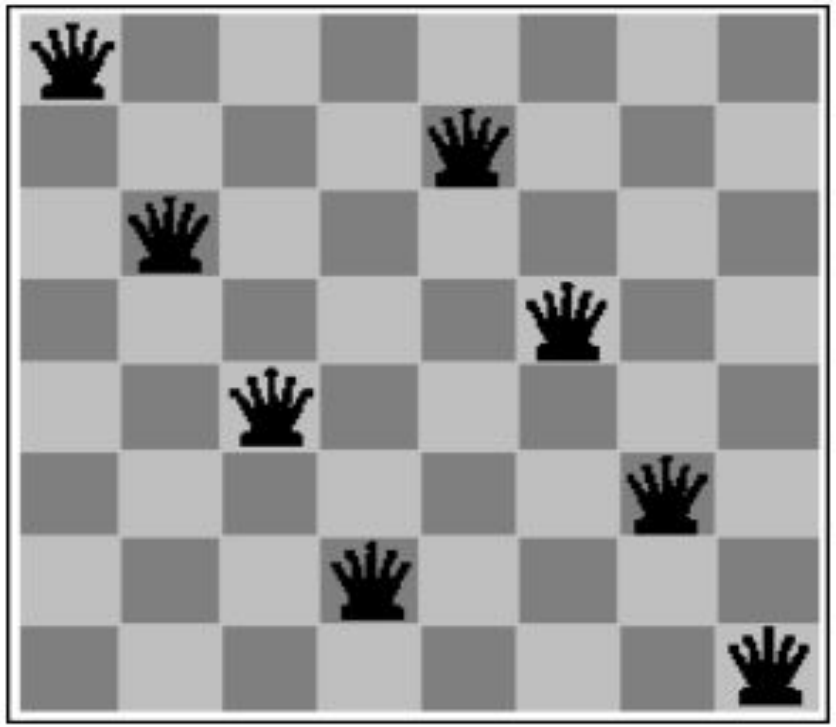
Solution:

Gallons in Four Jug	Gallons in Three Jug	Rules Applied
0	0	-
0	3	2
3	0	7
3	3	2
4	2	11
0	2	3
2	0	10

*(**Fig. 2.3** One Solution to the Water Jug Problem)*

8-queens problem

- Statement: "Can you place 8 queens on a standard 8×8 chessboard so that no two queens threaten each other?"
- 1. Initial State: Any arrangement of 0 to 8 queens on board.
- 2. Operators: add a queen to any square. Add one queen in the next row such that it doesn't conflict with existing queens. From state [0, 4], next step tries to place queen in row 2 at columns 0–7 avoiding attacks.
- 3. Goal Test: 8 queens on board, none attacked. ***[0, 4, 7, 5, 2, 6, 1, 3]***
- 4. Path cost: not applicable or Zero (because only the final state counts, search cost might be of interest).
- State Space Size:
  - Theoretically: 88=16,777,2168^8 = 16,777,21688=16,777,216 possible configuration
  - But using row-by-row placement with checks reduces it drastically



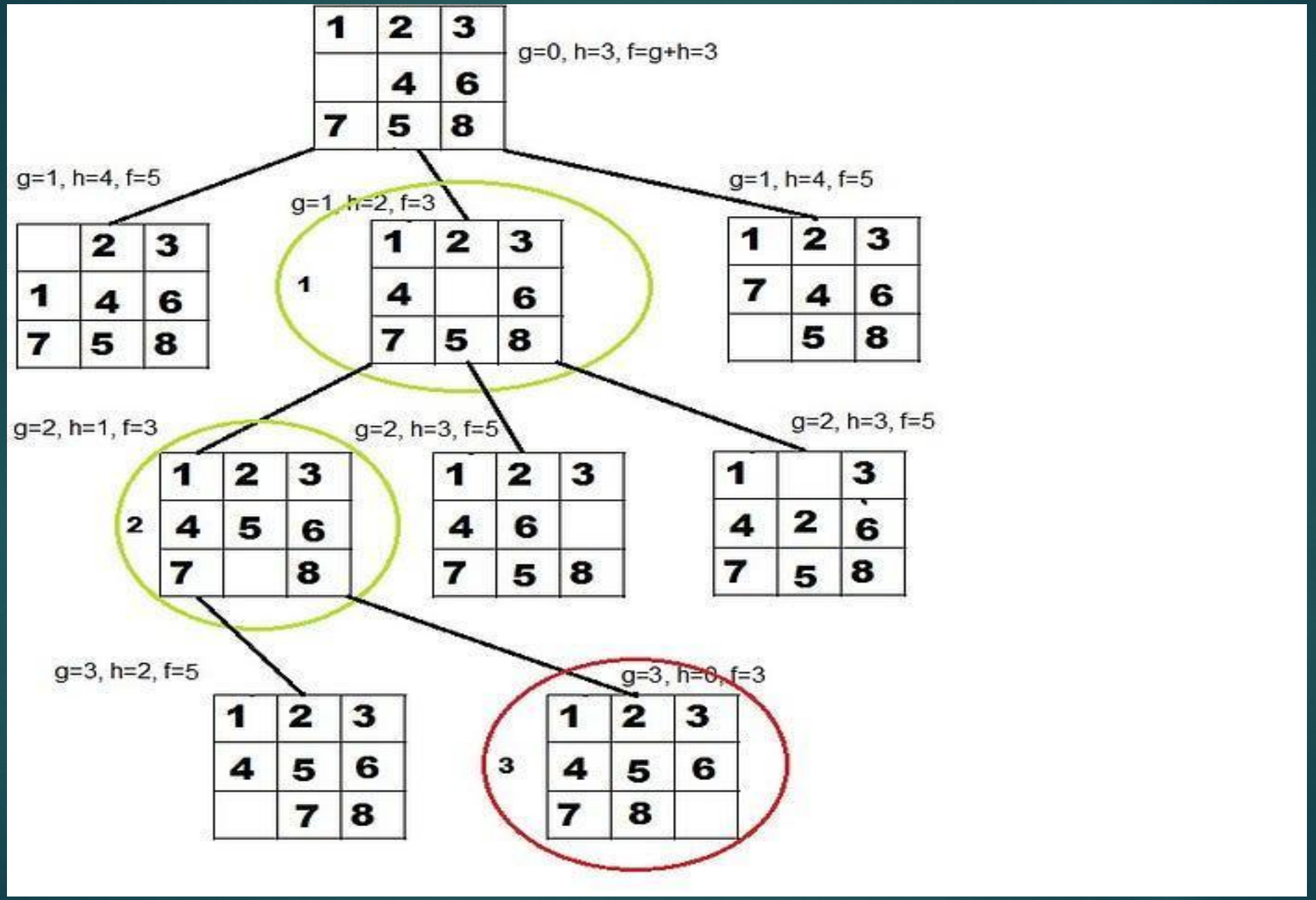
8-Puzzle Problem

- A 3×3 board with 8 numbered tiles and 1 empty space
- The goal is to **rearrange the tiles** from a **start configuration** to a **goal configuration** by sliding tiles into the empty space (represented as 0 or blank).

Initial state	Goal State:
1 2 3	**1 2 3**
4 0 6	**4 5 6**
7 5 8	**7 8 0**

- State Representation: Each state is a configuration of the board. It can be represented as: [1, 2, 3, 4, 0, 6, 7, 5, 8]
- Operators (Actions) and State Space size: Total possible arrangements = 9! = 362,880

  - **Up**
  - **Down**
  - **Left**
  - **Right**


1 2 3
4 6
7 5 8
g=0, h=3, f=g+h=3
g=1, h=4, f=5
2 3
1 4 6
7 5 8
g=1, h=2, f=3
1
1 2 3
4 6
7 5 8
g=1, h=4, f=5
1 2 3
7 4 6
5 8
g=2, h=1, f=3
2
1 2 3
4 5 6
7 8
g=2, h=3, f=5
1 2 3
4 6
7 5 8
g=2, h=3, f=5
1 3
4 2 6
7 5 8
g=3, h=2, f=5
1 2 3
4 5 6
7 8
g=3, h=0, f=3
3
1 2 3
4 5 6
7 8

Control strategies

- Control Strategies means how to decide which rule to apply next during the process of searching for a solution to a problem:

1. **It should cause motion.** In water jug problem, if we apply a simple control strategy of starting each time from the top of rule list and choose the first applicable one, then we will never move towards solution.

2. **It should explore the solution space in a systematic manner.** If we choose another control strategy, let us say, choose a rule randomly from the applicable rules then definitely it causes motion and eventually will lead to a solution. But one may arrive to same state several times. This is because control strategy is not systematic.

Properties of Search Algorithms

1. **Completeness** – Is a solution guaranteed to be found if atl east one solution exists?

2. **Optimality** – Is the solution found guaranteed to be the best (or lowest cost) solution if there exists more than one solution?

3**. Time Complexity** – The upper bound on the time required to find a solution, as a function of the complexity of the problem.

4. **Space Complexity** – The upper bound on the storage space (memory) required at any point during the search, as a function of the complexity of the problem

PRODUCTION SYSTEMS

Production systems provide appropriate structures for performing and describing search processes. A production system has four basic components as enumerated below:

- A set of rules each consisting of a left side that determines the applicability of the rule and a right side that describes the operation to be performed if the rule is applied.
- A database of current facts established during the process of inference.
- A control strategy that specifies the order in which the rules will be compared with facts in the database and also specifies how to resolve conflicts in selection of several rules or selection of more facts.
- A rule applier

The production rules operate on the knowledge database. Each rule has a precondition—that is, either satisfied or not by the knowledge database. If the precondition is satisfied, the rule can be applied. Application of the rule changes the knowledge database. The control system chooses which applicable rule should be applied and ceases computation when a termination condition on the knowledge database is satisfied.

CHARACTERISTICS OF PRODUCTION SYSTEMS

1. A monotonic production system is a production system in which the application of a rule never prevents the later application of another rule that could also have been applied at the time the first rule was selected.
2. A non-monotonic production system is one in which this is not true.
3. A partially communicative production system is a production system with the property that if the application of a particular sequence of rules transforms state P into state Q, then any combination of those rules that is allowable also transforms state P into state Q.
4. A commutative production system is a production system that is both monotonic and partially commutative.

Table 2.1 Four Categories of Production Systems

Production System	Monotonic	Non-monotonic
Partially Commutative	Theorem Proving	Robot Navigation
Non-partially Commutative	Chemical Synthesis	Bridge

Issues in the Design of Search Programs

- Each search process can be considered to be a tree traversal. The object of the search is to find a path from the initial state to a goal state using a tree. The number of nodes generated might be huge; and in practice many of the nodes would not be needed. The secret of a good search routine is to generate only those nodes that are likely to be useful, rather than having a precise tree. The rules are used to represent the tree implicitly and only to create nodes explicitly if they are actually to be of use.

The following issues arise when searching:

• The tree can be searched forward from the initial node to the goal state or backwards from the goal state to the initial state.

• To select applicable rules, it is critical to have an efficient procedure for matching rules against states.

• How to represent each node of the search process? This is the knowledge representation problem or the frame problem.